

1) Discuss the synchronization issues that may arise and justify the mechanisms you selected to address them.

- The first synchronization issue we faced occurs when producers and consumers try to access the circular buffer simultaneously. To prevent producers from overwriting data and consumers from reading empty slots, we protected the buffer by a mutex lock (*lock* in the *CircularBuffer* struct) and two condition variables (*not_empty* and *not_full* from the struct) to manage the state of the buffer.
- The second issue we found came from the shared file reading. We have an *offset* global variable, protected by another lock (*offsetLock*), which prevents threads from reading the same blocks of data when calling *lseek()* and *read()*.
- The third synchronization issue involved the histogram. We initialized it as a global variable *histogram* protected by *histLock* to avoid passing it as a parameter to the consumers, and for each consumer, we created a local histogram. This way, we only need to use the histogram's lock once per consumer thread at the end of its execution.
- Finally, we had to manage the termination state. To do so, we used a producer counter *active_producers* and a flag *producers_finished*, both protected by *termLock*. This ensures that all threads know if the producers have finished or not.

2) Explain why your solution prevents race conditions, starvation, and busy waiting.

- Race conditions: To avoid them, we protected every shared memory location with mutexes as explained above. This ensures that only one thread can modify them at a time and avoids data loss.
- Starvation: We used *pthread_cond_broadcast()* to prevent starvation when the last producer finishes, this ensures that all blocked consumers are woken up to check for the exit condition (flag), rather than just one (none left behind).
- Busy Waiting: Lastly, we avoided wasting CPU cycles by using *pthread_cond_wait()*, which puts thread into a waiting stage (sleep) until signaled by another thread, thus consuming zero CPU cycles.

3) Describe how the program behaves since the first producer reaches EOF until the process ends, paying special attention to consumers that may still be blocked waiting for data. Note that when a producer reaches the EOF, there might be other producers which are still doing them.

- When the first producer reaches EOF, it decrements *active_producers* under *termLock*, then terminates while the others producers continue independently, and consumers keep extracting blocks from the buffer.
- When the last producer reaches EOF, it decrements *active_producers* to zero, and sets the *producers_finished* flag to 1. Then, it 'wakes up' all waiting consumers by calling *pthread_cond_broadcast(&cb.not_empty)*.
- At this point, consumers will wake up, re-evaluate their waiting condition, see that *producers_finished* is true and break their loops.
- The remaining consumers will finish their executing, merging all local histograms into the global one and terminate.
- Lastly, the main, which was waiting for all threads to finish, resumes from *pthread_join()*, writes the results into the output file and frees all the remaining allocated resources.